Integrating Test Suite Minimization into LASSO

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk
laith.sandouk@students.uni-mannheim.de
born May 15th, 2005
in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: http://swt.informatik.uni-mannheim.de

Abstract

Software testing has become increasingly resource-intensive, often consuming nearly half of a software system's development budget [1]. Large-scale test suites in modern projects can include thousands of automatically generated test cases, taking days or even weeks to execute [5]. This escalating complexity and cost of testing demand effective strategies to maintain manageable, high-impact test suites. *Test suite minimization* offers a solution by eliminating redundant tests to reduce execution time and maintenance overhead [6]. However, naïve reduction of test suites can risk omitting tests that detect faults, potentially undermining software quality [7].

The LASSO platform (Large-Scale Software Observatorium) at the University of Mannheim provides comprehensive support for automated large-scale code analysis and testing in Java [3], including the generation of extensive test suites. Currently, LASSO lacks a built-in mechanism for test suite minimization, leaving an efficiency gap in its otherwise powerful testing pipeline. This thesis addresses that gap by investigating state-of-the-art Java-based test suite minimization techniques and integrating the most effective solution into LASSO. We survey existing approaches—coverage-based, fault-detection-preserving, and heuristic methods (e.g., greedy algorithms, clustering)—and evaluate them on real-world test suites in terms of reduction ratio and coverage preservation. Based on this analysis, we implement the best-performing minimization technique within LASSO.

The contributions of this work include a systematic evaluation of test suite minimization tools, empirical benchmarks of their impact on test suite size and coverage, and the extension of the LASSO platform with an integrated minimization capability to enhance its testing efficiency.

Contents

List of Figures

List of Tables

List of Abbreviations

LASSO	Large-Scale Software Observatorium
LSL	LASSO Scripting Language
OWL	Web Ontology Language
TC	Telecommunications Company

1. Introduction

1.1. Background and Motivation

Quality assurance through software testing is increasingly challenging as software systems grow in scale and complexity. Testing activities can require substantial time and resources—often accounting for 40–50% of total development costs [1]. As projects evolve, their test suites tend to expand cumulatively [5]. Prior studies have observed that regression test suites for enterprise systems may run for days or weeks [5], delaying feedback in continuous integration and increasing the cost of each testing cycle.

Automated test generation techniques exacerbate this issue by producing many overlapping or redundant tests. This leads to bloated test suites that are harder to maintain while adding little new coverage [7]. Industry standards such as ISO/IEC/IEEE 29119 [2] emphasize systematic test design and maintenance, highlighting the need for techniques like *test suite minimization* [6]. Minimization aims to remove redundant tests while preserving coverage and fault-detection effectiveness. Since finding the optimal minimal subset is NP-complete [6], practical approaches rely on heuristics such as greedy set-cover approximations, clustering, or machine learning [7, 4].

Empirical studies show the trade-offs involved: Noemmer and Haas [4] found that minimization can reduce suite size by 70% while incurring an average 12.5% loss in fault detection. This balance between efficiency and effectiveness underscores the importance of careful selection of minimization techniques.

1.2. LASSO Context

The **LASSO** platform (Large-Scale Software Observatorium) was developed at the University of Mannheim to support scalable software analysis and experimentation [3]. It integrates static and dynamic analysis at scale, supports automated

test generation, and provides a domain-specific language (LSL) for orchestrating analysis pipelines. Despite these capabilities, LASSO currently lacks automated test suite minimization. As a result, generated test suites may grow excessively large, undermining LASSO's efficiency. Integrating minimization techniques directly into LASSO will reduce redundant test execution and improve throughput in large-scale testing workflows.

1.3. Problem Statement

While LASSO provides robust test generation and analysis, it lacks functionality for reducing redundant test suites. This leads to:

- Increased execution times,
- Higher computational and storage costs,
- Maintenance overhead, and
- Reduced efficiency of continuous integration workflows.

This thesis investigates how test suite minimization can be integrated into LASSO to address these issues.

1.4. Research Objectives

The objectives of this thesis are:

1. To survey state-of-the-art test suite minimization techniques and tools for Java.
2. To benchmark these techniques on real-world Java projects, measuring reduction ratio, coverage, and fault-detection preservation.
3. To select the best-performing approach based on empirical results.
4. To integrate this approach into the LASSO platform's pipeline.
5. To evaluate the integration's impact on LASSO's efficiency and effectiveness.

1.5. Research Questions

1. What test suite minimization techniques and tools are available for Java, and what are their strengths and limitations?
2. How do these techniques perform in terms of reduction ratio, coverage preservation, and fault-detection effectiveness?
3. Which approach provides the best trade-off between efficiency and effectiveness for integration into LASSO?
4. What are the benefits and drawbacks of integrating minimization into LASSO's workflow?

1.6. Thesis Structure

This thesis is structured as follows:

- **Chapter 2:** Literature review of test suite minimization approaches and tools.
- **Chapter 3:** Methodology and experimental design for benchmarking minimization techniques.
- **Chapter 4:** Design and implementation of the integration into LASSO.
- **Chapter 5:** Evaluation of the integration and analysis of results.
- **Chapter 6:** Conclusion and outlook for future work.

2. Implementation Requirements

2.1. Header Text

All software (i.e. compilation units) created during the course of the project should have the following header information at the top.

---

3. Conclusion

Every thesis should start with an Introduction section and end with a Conclusion section. The other sections in between should be adapted to the specific topic of the thesis.

Bibliography

[1] Barry W. Boehm. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981.

[2] *ISO/IEC/IEEE 29119:2013 – Software and Systems Engineering – Software Testing*. International Organization for Standardization, 2013.

[3] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.

[4] Robert Noemmer and Rainer Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Proceedings of the Software Quality Days (SWQD)*. Springer, 2020, pp. 89–104.

[5] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[6] W. Eric Wong et al. „Effect of test set minimization on fault detection effectiveness“. In: *Proceedings of the 17th International Conference on Software Engineering (ICSE)*. ACM, 1995, pp. 41–50.

[7] Shin Yoo and Mark Harman. „Regression testing minimization, selection and prioritization: A survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120.

Appendix

A.1. Example Appendix

This is a sample appendix entry.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, September 8, 2025

Max Mustermann

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, September 8, 2025

Max Mustermann